

AI TestGen: Marketplace Extension for Auto Test Generation & Coverage Gating

Title

AI TestGen — GitHub Action & Azure DevOps Task for Intelligent Test Generation, Coverage Gating, and CI Reporting

Context / Background

Teams want AI to write practical tests, but one-off scripts don't scale across repos and pipelines. This project creates a **publishable marketplace extension** (GitHub Action + Azure DevOps task) that any team can drop into CI to: analyze changed code, generate meaningful tests, run them, measure coverage, and **gate merges**—with clear, auditable outputs.

Summary (What you'll build)

Ship a **production-grade CI extension** that (1) analyzes target code (preferably changed files), (2) generates **unit/integration/E2E tests** via AST + optional LLM, (3) runs tests and computes **coverage deltas**, (4) validates usefulness by catching seeded bugs (mutation or seed-faults), and (5) **publishes results back to CI** (coverage, artifacts, SARIF, PR comments). Deliver **both** a GitHub Action and an Azure DevOps task with the **same interface** and behavior.

Objectives / Goals (Success Criteria)

- 1 Provide a **single command** CI step (Action/Task) with clear inputs/outputs and semver tags.
- 2 Support AST-only mode (offline) and **AI-assisted mode** (OpenAI/Azure OpenAI/Anthropic—vendor-agnostic).
- 3 Generate tests for at least **one language end-to-end** (start with Python/pytest or JS/Jest), with hooks for Java/C#.
- 4 Execute tests, compute coverage, and **gate** PRs by thresholds (e.g., fail if coverage ↓ or < min).

- 5 Validate usefulness: kill rate against **seeded bugs** (or mutation score) with a pass/fail policy.
- 6 Publish CI artifacts (generated tests, coverage reports, JSON summary, optional SARIF) and **PR comment** summary.
- 7 Package for marketplace: versioned release, README, examples, and security notes.

Acceptance targets (minimum):

- Coverage **delta $\geq$ +5pp** on sample repos OR maintain $\geq$ configured threshold.
- **Bug-kill rate $\geq$ 30%** on seeded faults for the demo repo.
- Action/Task **runs in < 10 min** on RealWorld sample (changed-files scope).
- Clean failure modes (no secrets in logs, descriptive exit codes).

Scope Clarifications

In-scope: GitHub Action + Azure DevOps task, one primary language end-to-end (e.g., Python or JS), AST + LLM hybrid, coverage gating, mutation/seed fault validation, PR outputs.

Out-of-scope: Full polyglot parity; flaky-test elimination; IDE plugins.

Tech / Tooling Expectations

Area	Preferred Tooling (or equivalent)
Code analysis	AST libs / tree-sitter; static call graph where possible
AI layer (opt-in)	Any LLM provider (OpenAI / Azure OpenAI / Anthropic). Must be toggleable.
Test frameworks	Start with pytest or Jest (expand later to JUnit/xUnit)
Validation	Seeded bug injection or mutation testing (mutmut / Stryker)

CI packaging **GitHub Action** (node16 or docker action) + **Azure DevOps task** (tfx)

Reports JSON summary + LCOV + optional SARIF; PR comment renderer

Security OIDC to LLM where possible, **no code exfiltration**, redact secrets

Inputs & Outputs (Contract)

Inputs (common across GH/ADO)

- language: python|javascript|java|csharp (start with one)
- mode: ast (default) | ai | hybrid
- exclude: glob(s)
- coverage_threshold: integer (e.g., 80)
- delta_threshold: integer (pp change to enforce, e.g., +5)
- bug_validation: seeded|mutation|off (default: seeded)
- llm_provider: openai|azure_openai|anthropic|none
- llm_model: string (optional)
- llm_api_key: from secret store (never plain input in examples)
- max_cost_usd: cap (soft stop)
- fail_on_no_change: true|false (default false)
- report_dir: path for artifacts (default .testgen-reports)

Outputs

- generated_tests_count
- coverage_before, coverage_after, coverage_delta

- bug_kill_rate (or mutation_score)
- report_path (JSON), lcov_path (LCOV)
- sarif_path (optional)
- summary (short text)

Usage Examples

GitHub Action (action.yml snippet & workflow)

```
# action.yml (metadata)
name: 'AI TestGen'
description: 'Generate tests with AST/LLM, run coverage, gate PRs'
inputs:
  language: { required: true }
  mode: { required: false, default: 'ast' }
  coverage_threshold: { required: false, default: '80' }
  delta_threshold: { required: false, default: '5' }
  bug_validation: { required: false, default: 'seeded' }
  llm_provider: { required: false, default: 'none' }
  llm_model: { required: false }
  max_cost_usd: { required: false, default: '0.50' }
  report_dir: { required: false, default: '.testgen-reports' }
outputs:
  generated_tests_count: {}
  coverage_before: {}
  coverage_after: {}
  coverage_delta: {}
  bug_kill_rate: {}
runs:
  using: 'node16' # or docker
  main: 'dist/index.js'

# .github/workflows/ci.yml
jobs:
  testgen:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      pull-requests: write
    steps:
```

- uses: actions/checkout@v4
- uses: tj-actions/changed-files@v45
 - id: cf
- uses: your-org/ai-testgen-action@v1
 - id: gen
 - with:
 - language: python
 - mode: hybrid
 - coverage_threshold: 80
 - delta_threshold: 5
 - bug_validation: seeded
 - llm_provider: openai
 - llm_model: gpt-4o-mini
 - env:
 - OPENAI_API_KEY: \${ secrets.OPENAI_API_KEY }
- name: PR Comment
 - if: \${ github.event_name == 'pull_request' }
 - uses: marocchino/sticky-pull-request-comment@v2
 - with:
 - header: AI TestGen
 - message: |
 - **AI TestGen Summary****
 - Generated tests: \${ steps.gen.outputs.generated_tests_count }
 - Coverage: \${ steps.gen.outputs.coverage_before }% → \${ steps.gen.outputs.coverage_after }% (**Δ \${ steps.gen.outputs.coverage_delta }pp**)
 - Bug kill rate: \${ steps.gen.outputs.bug_kill_rate }%
 - Reports: \${ steps.gen.outputs.report_path }

Azure DevOps (task.json usage)

```
# azure-pipelines.yml
steps:
- checkout: self
- task: AITestGen@1
  inputs:
    language: 'python'
    mode: 'hybrid'
    coverage_threshold: '80'
    delta_threshold: '5'
    bug_validation: 'seeded'
    llm_provider: 'azure_openai'
    llm_model: 'gpt-4o-mini'
  env:
    AZURE_OPENAI_API_KEY: $(AZURE_OPENAI_API_KEY)
```

Deliverables

1. **GitHub Action** repo (with action.yml, code, README, examples, semver tags).
2. **Azure DevOps Extension** (task implementation, task.json, vss-extension.json, packaged via tfx).
3. **TestGen Engine** (library) with: analyzers, generators, runners, coverage collectors, validators.
4. **Reports & Artifacts**: JSON summary, LCOV/HTML, seeded-bug results, optional SARIF.
5. **Docs**: setup guides, inputs/outputs reference, security notes, cost controls, troubleshooting.
6. **Demo pipelines** against RealWorld repos (frontend + backend) showing changed-files flow and gating.

Evaluation Criteria (weights)

- **Quality & Relevance of Generated Tests (30%)** – Readable, maintainable, catch real faults.
- **Integration Effectiveness (25%)** – One-step CI usage, clear artifacts, PR comments, stable exits.
- **Coverage Improvement (20%)** – Demonstrated delta or threshold adherence.
- **Bug Detection (15%)** – Seeded/mutation “kill rate.”
- **Performance & Scalability (5%)** – Reasonable runtime, changed-files optimization.
- **Documentation & Usability (5%)** – Marketplace-ready README, examples, support notes.

12 – Guidelines & Best Practices

- **AI safety & governance**: opt-in AI; redact secrets; cap spend (max_cost_usd); log prompts offline only (no source exfiltration).
- **Reproducibility**: pin dependencies; deterministic AST generation; snapshot tests for prompts.
- **Security**: OIDC to provider where possible; never print secrets; least-privilege tokens.
- **DX**: dry-run mode (no_write); verbose logs toggle; graceful fallbacks to AST if AI unavailable.
- **Scalability**: prioritize **changed files**; cache AST/embeddings between steps.

Marketplace Checklist (shipping)

GitHub: public repo, action.yml, semver release tag (v1), README (inputs/outputs, examples), logo, Marketplace listing.

Azure DevOps: package with tfx-cli, vss-extension.json, task icon, publisher ID, listing text, samples; verify YAML snippet works.